

Chapter 1

Tables

Why Organize Data in Tables?

- The need to **organize information** existed long before computers and databases. Early civilizations kept records of important information such as?
 - **taxes, property, trade, crops, livestock, and population.**
- Ancient record keepers used clay tablets, papyrus, and later paper ledgers to maintain these records.
- Information could be written in a **free-form style**, such as?
 - Marcus paid 20 coins in taxes and owns 3 acres. Julia owns 5 acres of land and paid 25 coins. Antonius paid 15 coins in taxes and owns only 2 acres.
- While this free-form approach may be understandable for a small amount of information, it becomes increasingly difficult to manage as hundreds or thousands of records accumulate. Finding a particular person, comparing values, calculating totals, or identifying missing information could require searching through large amounts of text. **What do you think people could have done to organize this information more effectively?**

Earlier civilizations recognized that a **table** provides a much more organized approach:

Person	Tax Paid	Land Owned
Marcus	20 coins	3 acres
Julia	25 coins	5 acres
Antonius	15 coins	2 acres

File-Based Systems and Tables

- Tables organize information into **rows and columns**. Each row can represent one person, item, or transaction, while each column represents the same type of information for every record.
- What are some advantages of using a table?
 - Tables data easier to **read, compare, sort, search, count, and summarize**.

What is a table?

- A **table** is a structured collection of related data organized into rows and columns. Each table represents a single entity (such as Customer information, Students, Employees, etc), where each row represents one **record** and each column represents a specific **attribute** or **field** of that record.
- Tables provide an organized way to store, retrieve, update, and manage data efficiently while maintaining relationships with other tables through keys.

Imagine the savings account for customers. What data do you think are relevant to store for each customer?

Table

Why is it a good idea to have a Customer ID field?

Attribute or Field

Record

Customer ID	Name	Address	Phone	Balance
1001	John Smith	125 Oak Street, Aurora, IL 60505	(630) 555-0101	\$ 1,005.00
1002	Michael Johnson	842 Maple Avenue, Naperville, IL 60540	(630) 555-0102	\$ 2,034.00
1003	Sophia Martinez	310 Pine Lane, Elgin, IL 60120	(847) 555-0103	\$ 5,340.00
1004	Daniel Kim	56 Cedar Court, Schaumburg, IL 60173	(847) 555-0104	\$ 3,432.00
1005	Olivia Brown	910 Birch Drive, St. Charles, IL 60174	(630) 555-0105	\$ 334.00
1006	Ethan Wilson	415 Willow Road, Geneva, IL 60134	(630) 555-0106	\$ 983.00
1007	Jane Smith	221 Elm Street, Batavia, IL 60510	(630) 555-0107	\$ 98.00
1008	Noah Garcia	780 Aspen Circle, Bartlett, IL 60103	(630) 555-0108	\$ 123.00
1009	Isabella Davis	144 Hickory Way, Streamwood, IL 60107	(630) 555-0109	\$ 12,345.00
1010	Liam Anderson	633 Spruce Avenue, Plainfield, IL 60544	(815) 555-0110	\$ 234.00

Tools for Creating and Managing Table-Based Data

Data can be stored in several file formats that organize information into **rows and columns**. These files can be created and edited using different software tools.

Can you think of different apps we use to create a table?

- **Microsoft Excel** is one of the most commonly used tools for creating tables. It primarily uses the **.xlsx** format, while older versions of Excel used **.xls**. Excel supports formulas, formatting, charts, multiple worksheets, and other features for organizing and analyzing data. A **.csv** file stores data as plain text, with values usually separated by commas.
- **Google Sheets** is a web-based spreadsheet application. Like Excel, it allows users to create and manipulate data in rows and columns. Google Sheets files can be downloaded or exported as **.xlsx**, **.csv**, and several other formats.
- **LibreOffice Calc** is a free and open-source spreadsheet application. Its standard spreadsheet format is **.ods**, but it can also open and save files in formats such as **.xlsx**, **.xls**, and **.csv**. It provides many of the same basic spreadsheet features as Microsoft Excel.

Possible problems with using tables

- Although file-based tables are relatively simple to create and use, problems can arise as the amount of data grows and multiple files need to share related information.
- These problems can include:
 - **data redundancy, data inconsistency, data isolation, and difficulty maintaining relationships between data.**

Data Redundancy

- Imagine the savings, checking and loan account for customers.

- **Savings Account File**

- Customer ID: 1001
- Name: John Smith
- Address: 123 Main St.
- Phone: 555-1234
- Balance: \$5,200

- **Checking Account File**

- Customer ID: 1001
- Name: John Smith
- Address: 123 Main St.
- Phone: 555-1234
- Balance: \$1,850

- **Loan File**

- Customer ID: 1001
- Name: John Smith
- Address: 123 Main St.
- Phone: 555-1234
- Loan Balance: \$15,000

- **Why is this a potential source of a problem?**

Information is duplicated across multiple files.

If John Smith moves to a new address, the bank must update **every file** containing his address. If one file is updated but another is not, the bank ends up with conflicting information.

Data redundancy is a situation that occurs in a database when a field needs to be updated in more than one table.

Data Inconsistency

- Suppose a customer, **John Smith**, changes his address from **123 Main St.** to **456 Oak Ave.** Because the bank uses a file-based system, his information is stored separately in the:
 - **Savings Account File** → Address updated to **456 Oak Ave.** 
 - **Checking Account File** → Still **123 Main St.** 
 - **Loan File** → Address updated to **456 Oak Ave.** 
- Now the bank has **conflicting information** about the same customer. One department may send statements to the old address while another sends them to the new address.

This is an integrity problem because the data is no longer accurate and consistent across the system.

A **Data Inconsistency** occurs when the accuracy or consistency of data is compromised, resulting in conflicting or incorrect information.

Data Redundancy is not automatically bad.

But **unnecessary or uncontrolled data redundancy is a problem.**

For example, suppose the department name Information Technology is repeated for 500 employees in a single Employee table:

That redundancy creates several potential problems:

- **Wasted storage** — the same information is stored repeatedly. This is less important today for small data, but can matter at scale.
- **Update problems** — if the department changes its name to Technology Services, you may need to update hundreds of records.
- **Inconsistency risk** — some records might be updated while others are not.
- **Maintenance overhead** — the same fact has to be maintained in multiple places.

EmployeeID	Name	Department
1001	Emily	Information Technology
1002	Michael	Information Technology
1003	Sophia	Information Technology
...	...	...

- A database design would typically separate this into:
 - **EMPLOYEE**
 - **DEPARTMENT**
- Now the fact that Department 2 is called Information Technology is stored **once**.

EmployeeID	Name	DepartmentID
1001	Emily	2
1002	Michael	2
1003	Sophia	2

DepartmentID	DepartmentName
2	Information Technology

Data Isolation

- Suppose a school uses separate files:

STUDENT file

StudentID	Name	Major
101	Maria Cruz	Nursing
102	John Smith	Biology

PAYMENT file

StudentNo	Amount	Date
101	\$500	8/20/2026
102	\$300	8/21/2026

• Why is this a problem?

If a new application needs to display **each student's name together with their payments**, it must know:

- which files contain the required information,
- how each file is structured,
- that **StudentID** in one file corresponds to **StudentNo** in another, and
- how to combine the records correctly.

In a traditional file-based system, these relationships are often handled by **application programs rather than centrally by a DBMS**. As the number of files and applications increases, retrieving and combining related data becomes increasingly difficult.

Data isolation occurs when the data exists, but because it is scattered and isolated in different files, a new program may have difficulty finding and combining the data it needs.

Another problem with Data Isolation

Suppose two separate files/tables store the same date differently:

- Customers.csv → 08/29/2026
- Orders.csv → 2026-08-29
- Another system → 29-Aug-2026

The **data isolation** problem is that related data is stored in **different files, formats, or systems**, making it harder for an application to retrieve, combine, and process the data.

Data Security

- Suppose a bank teller's job is to process deposits and withdrawals for **checking and savings accounts** only. However, because the file-based system has poor access controls, the teller can also open the **Loan File**.
- As a result, the teller can view confidential information such as:
 - Customers' loan balances
 - Credit history
 - Monthly loan payments
- Why is this a problem?
- Even though this information is unrelated to the teller's job, they still have access to it.
- A **security problem** occurs when unauthorized users are able to access, view, modify, or delete data they should not have permission to access.

Data Concurrency

- Suppose **John Smith** has **\$5,000** in his **checking account**.
- At the same time:
 - A **bank teller** is processing a **\$1,000 cash withdrawal**.
 - An **ATM** is processing a **\$500 withdrawal** from the same checking account.
 - Both transactions read the balance before either one updates it.
 - The teller reads the balance: **\$5,000**.
 - The ATM also reads the balance: **\$5,000**.
 - The teller subtracts \$1,000 and saves the new balance: **\$4,000**.
 - The ATM subtracts \$500 from the balance it originally read (\$5,000) and saves **\$4,500**.
 - The final balance becomes **\$4,500**, when it should be **\$3,500**.
- This happened because **both transactions accessed and updated the same account simultaneously**, causing one update to overwrite the other.
- A **concurrency problem** occurs when two or more users or transactions access and modify the same data at the same time, resulting in incorrect or inconsistent data.

So, to summarize, the issues with maintaining a file-based system are:

1. **Data Redundancy**- is a situation that occurs in a database when a field needs to be updated in more than one table.
2. **Data Inconsistency**- occurs when the accuracy or consistency of data is compromised, resulting in conflicting or incorrect information.
3. **Data Isolation** - occurs when related data is stored in separate files or systems, making it difficult to access, combine, and analyze the information.
4. **Security problem** - occurs when unauthorized users are able to access, view, modify, or delete data they should not have permission to access.
5. **Concurrency problem** - occurs when two or more users or transactions access and modify the same data at the same time, resulting in incorrect or inconsistent data.

File-system approach:

Application Program → Individual Files

Database approach:

Application Program → DBMS → Shared Database